

Don't overdo it with use of assertions for everything in your code. Check for only the conditions that are necessary for correct functioning of the code. If your code is working normally even when a particular assertion is false, you can safely remove the assertion.

Introduce Assertion

مشکل (Problem)

برای اینکه بخشی از کد درست کار کنه، باید شرایط یا مقادیر خاصی برقرار باشن.

راه حل (Solution)

جایگزین کن **assertion** این پیش فرض ها رو با بررسی های

مثال از صفحه

قبل:

```
csharp

double GetExpenseLimit()
{
    // Should have either expense limit or a primary project.
    return (expenseLimit != NULL_EXPENSE) ?
        expenseLimit :
        primaryProject.GetMemberExpenseLimit();
}
```

بعد:

```
csharp

double GetExpenseLimit()
{
    Debug.Assert(expenseLimit != NULL_EXPENSE || primaryProject != null);

    return (expenseLimit != NULL_EXPENSE) ?
        expenseLimit :
        primaryProject.GetMemberExpenseLimit();
}
```

چرا ریفکتور کنیم؟ (Why Refactor)

توضیح	دلیل
فرضیات کد رو آشکار کن	مستندسازی زنده
اگه فرض اشتباه باشه، قبل از فاجعه متوقف میشه	پیدا کردن باگ
کامنت‌هایی که شرایط کار متد رو توضیح میدن ← جاهای مناسب برای assertion	راهنما

مزایا (Benefits)

اگه به پیش‌فرض درست نباشه و کد نتیجه اشتباه بده، بهتره قبل از عواقب مرگبار و خرابی داده‌ها اجرا متوقف بشه.

نقطه ضعف (Drawbacks)

Exception	Assertion	
خطای کاربر یا سیستم (قابل مدیریت)	باگ برنامه‌نویس (نباید اتفاق بیفته)	کی استفاده کنی
کرد catch میشه	کرد handle نمیشه	مدیریت

استفاده **Exception** اگه خطا می‌تونه توسط کاربر یا سیستم ایجاد بشه و می‌تونی مدیریتش کنی، از `assertion`، کن. در غیر این صورت

چطور انجام بدیم (How to Refactor)

توضیح	قدم
اضافه کن <code>assertion</code> ، وقتی دیدی یه شرط فرض شده	۱
نباید رفتار برنامه رو تغییر بده <code>assertion</code> اضافه کردن	۲
زیاده‌روی نکن! فقط شرایطی که برای کارکرد درست کد ضروری هستن رو چک کن	۳

C# مثال ساده

```
csharp

public void TransferMoney(Account from, Account to, decimal amount)
{
    //پیش‌فرض: حساب‌ها معتبر هستن و مبلغ مثبته
    Debug.Assert(from != null, "حساب مبدأ نباید null باشد");
    Debug.Assert(to != null, "حساب مقصد نباید null باشد");
    Debug.Assert(amount > 0, "مبلغ باید مثبت باشد");
}
```

```

        from.Balance -= amount;
        to.Balance += amount;
    }

```

🔥 تمرین

ریفکتور کن **Introduce Assertion** این کد رو با

csharp

```

public class Calculator
{
    // فرض: مقسوم‌علیه هیچوقت صفر نیست
    // فرض: هر دو عدد مثبت هستند
    public double Divide(double a, double b)
    {
        return a / b;
    }

    // فرض: آرایه هیچوقت خالی نیست
    // فرض: null آرایه نیست
    public double GetAverage(int[] numbers)
    {
        int sum = 0;
        foreach (int n in numbers)
            sum += n;
        return (double)sum / numbers.Length;
    }
}

```

(using System.Diagnostics;) .راهنمایی: از Debug.Assert استفاده کن

👉!بفرست تا بررسی کنیم

خودت حلش کن

✅ **Introduce Assertion** کد اصلاح شده با

csharp

```

using System.Diagnostics;

public class Calculator
{
    public double Divide(double a, double b)
    {
        Debug.Assert(b != 0, "مقسوم‌علیه نباید صفر باشد");
        Debug.Assert(a >= 0 && b >= 0, "اعداد باید مثبت باشند");

        return a / b;
    }
}

```

```
public double GetAverage(int[] numbers)
{
    Debug.Assert(numbers != null, "آرایه نباید null باشد");
    Debug.Assert(numbers.Length > 0, "آرایه نباید خالی باشد");

    int sum = 0;
    foreach (int n in numbers)
        sum += n;
    return (double)sum / numbers.Length;
}
```

 مقایسه

نسخه اصلاح	کد اصلی	
Assert آشکار با 	فقط توی کامنت یا ذهن برنامه نویس	پیش فرض های پنهان
 مشخص میشه Debug فوراً توی	خطا هنگام اجرا	تقسیم بر صفر
 مشخص میشه Debug فوراً توی	NullReferenceException	خالی/null آرایه
 ها حذف میشن Assert	-	Release تأثیر روی

 نکته مهم

```
csharp

Debug.Assert(condition); // اجرا میشه Debug فقط توی
Trace.Assert(condition); // اجرا میشه Debug و Release توی
```

 Introduce Assertion شد تموم!

یسته شد Comments، و با این:

وضعیت	درمان
	Extract Variable
 (قبلاً)	Extract Method
 (قبلاً)	Rename Method
	Introduce Assertion

متنش رو بفرست Duplicate Code  بریم